

Project Plan & Technical Specification

Adaptive Attention Computation: Balancing Efficiency and Retrieval in Long-Context Transformers

Team Size	Total Workload	Hardware Environment	Target Milestone
6 Engineers / Researchers	8.0 – 9.5 Hours per Person	Kaggle Free Tier (Dual T4, 16GB)	Empirical Results & 7-Slide Deck

Core Research Pivot

Initial benchmarks proved that basic kernelized linear attention achieves linear scaling (9.7 ms at N=65,536 vs 1,602 ms for SDPA), but collapses on key-value retrieval (14% accuracy at N=1,024 vs 100% for Softmax). This project pivots to **Adaptive Attention Computation**: dynamically allocating feature capacity based on input difficulty.

1. Decoupled Sprint Architecture (Zero-Bottleneck Workflow)

To eliminate idle waiting times across all 6 members, work is partitioned using **Contract-First Engineering**. Component interfaces are frozen in Hour 0 so members develop in parallel against mock implementations until a unified 30-minute merge in Hour 7.5.

Role	Core Responsibilities	Primary Output	Workload
Person 1 Theory Lead	Formulate adaptive rank equations; define input difficulty metric; build standalone <code>AdaptiveRankAttention</code> kernel.	<code>adaptive_kernel.py</code> + LaTeX math specification	9.0 Hours
Person 2 Compute Lead	Profile execution time, peak VRAM, and empirical scaling exponents on Kaggle T4 across N=1K to 64K.	<code>01_T4_Efficiency.ipynb</code> + latency/VRAM curves	9.5 Hours
Person 3 Retrieval Lead	Implement synthetic Passkey / Needle-In-A-Haystack suite; test Softmax, Linear, and FAVOR+ across depths.	<code>02_Passkey_Suite.ipynb</code> + 2D retrieval heatmaps	9.0 Hours
Person 4 Integration Lead	Patch Qwen2.5-0.5B attention blocks using mock and adaptive kernels; test generation and prefill latency.	<code>patch_qwen.py</code> + end-to-end forward test script	9.5 Hours
Person 5 Baselines Lead	Implement pure-PyTorch layer for Gated DeltaNet; benchmark throughput and state retention against linear attention.	<code>03_DeltaNet_T4.ipynb</code> + comparative metrics table	8.5 Hours
Person 6 Synthesis Lead	Standardize CSV schema; build automated chart generator; construct master 7-slide deck and final PDF report.	Master 7-Slide Deck + Consolidated PDF Document	9.5 Hours

2. Hour-by-Hour Master Execution Schedule

Hour Window	Person 1 (Theory)	Person 2 (Compute)	Person 3 (Retrieval)	Person 4 (Model Patch)	Person 5 (SOTA Baselines)	Person 6 (Deck & Lead)
0.0 – 1.0 h	Lock tensor I/O contract & rank formula	Build T4 memory tracking harness	Write Passkey token injection harness	Verify Qwen2.5 baseline inference	Extract Gated DeltaNet recurrent math	Lock CSV schema & setup presentation repo
1.0 – 3.0 h	Derive truncation error bounds	Benchmark Softmax vs SDPA (1K–64K)	Run baseline Softmax retrieval grid	Build <code>MockAttention</code> monkey-patch	Write standalone DeltaNet PyTorch layer	Build automated plotting script with mock data
3.0 – 5.5 h	Implement <code>AdaptiveRank</code> kernel	Benchmark ReLU+1 Linear Attention	Run Linear Attention Passkey sweep	Validate patched model forward pass	Profile DeltaNet latency on Kaggle T4	Draft slide narratives and speaker notes
5.5 – 7.5 h	Unit-test kernel for stability/NaNs	Fit scaling exponents (α) to CSV	Run FAVOR+ ($r=64$ – 1024) sweeps	Benchmark prefill latency on Qwen2.5	Profile state memory vs Linear attention	Ingest real CSVs; auto-render final charts
7.5 – 8.5 h	Hand kernel to P4; review math writeup	Clean Kaggle code & export plots	Export 2D heatmaps to presentation	Drop in P1's kernel into Qwen2.5	Finalize baseline comparative table	Assemble master slide deck & executive report
8.5 – 9.5 h	Rehearse theory Q&A for presentation	Verify Kaggle reproduction scripts	Audit retrieval figures in slide deck	Push patched model codebase	Audit comparative SOTA claims	Lead presentation dry-run & deliverable sign-off

3. Kaggle Free Tier (Tesla T4) Guardrails

- **FlashAttention-2 Workaround:** The Tesla T4 (SM 7.5) cannot run FlashAttention-2. Use PyTorch SDPA with memory-efficient math: `enable_flash=False, enable_mem_efficient=True`.
- **Batch Size Constraint:** Fix batch size to $B=1$ for all tests where sequence length $N \geq 8,192$ to prevent OOM errors.
- **Model Footprint:** Base model experiments are restricted to `Qwen/Qwen2.5-0.5B` (~1.0 GB VRAM in FP16), leaving >12 GB for KV caches and long context tensors.
- **Session Persistence:** For long evaluation sweeps ($N \geq 16,384$), submit notebooks via *Save & Run All (Commit)* to prevent 60-minute inactivity timeouts.

4. The 7-Slide Departmental Presentation Structure

Slide	Title	Core Content & Empirical Evidence	Lead Speaker
Slide 1	The Scaling Dilemma	Softmax quadratic bottleneck ($O(N^2)$) vs linear attention reordering ($O(N)$).	Person 6
Slide 2	Initial Hypothesis	Evaluating whether linear attention preserves retrieval quality under fair benchmarking.	Person 1
Slide 3	Experimental Design	Controlled setup: Tesla T4, float32/fp16, B=1, H=4, d=64, sequence lengths 1K to 64K.	Person 2
Slide 4	Empirical Findings	Efficiency won (9.7 ms vs 1602 ms), but retrieval collapsed (14% vs 100% at N=1024).	Person 3
Slide 5	The Real Research Gap	Information bottleneck: Compressing histories into fixed-size buffer causes key collisions.	Person 1
Slide 6	Proposed Architecture	Adaptive Attention Computation: Dynamic rank allocation ($r \in \{64, 256, 1024\}$) & hybrid routing.	Person 4
Slide 7	Research Roadmap	5-phase roadmap: SOTA baselines (DeltaNet) → Adaptive Kernel → Time-series testing (TiRex).	Person 5 / 6